



HAUTE ÉCOLE
D'INGÉNIERIE ET DE GESTION
DU CANTON DE VAUD
www.heig-vd.ch



Laboratoire 4 : Exploration de Xenomai 4 / EVL

Département : Technologies de l'Information et de la Communication (TIC)
Unité d'enseignement : Programmation Temps Réel (PTR)

Auteurs : Fabien Leger & Arnaut Leyre
Professeur : Brunet Yorick
Assistant : Miceli Jean-Pierre
Classe : PTR-A
Date : 19 mai 2026

1 Introduction

Ce laboratoire a pour objectif de découvrir plusieurs mécanismes de Xenomai 4 et EVL utilisés pour la programmation temps réel sous Linux. Il se concentre principalement sur la communication entre tâches temps réel et tâches Linux classiques, ainsi que sur la synchronisation de tâches périodiques.

Dans une première partie, les *cross-buffers* sont utilisés pour échanger des données entre une tâche EVL *out-of-band* et une tâche Linux *in-band*. Cela permet d'observer comment transmettre des informations sans perturber l'exécution temps réel.

La deuxième partie traite des *file proxies*. Ce mécanisme permet à une tâche temps réel d'envoyer des données vers un fichier de log sans effectuer directement des appels système Linux classiques. Dans ce laboratoire, il est utilisé pour enregistrer les valeurs simulées de deux capteurs.

Enfin, la dernière partie met en place un système de surveillance avec un *Watchdog* et des *EVL flags*. Le but est de vérifier que plusieurs tâches temps réel terminent correctement leur travail dans le temps prévu.

Ce laboratoire permet donc de mieux comprendre comment EVL facilite la communication, la synchronisation et la surveillance dans une application temps réel.

2 Cross-buffer

2.1 Mode normal

```

evl-de1 ~ # ./xbuf 20
Current: 4309266558270 ns | Delta current: 0 ns | Xbuf: 4309266497110 ns |
Delta xbuf: 0 ns
Current: 4309276562150 ns | Delta current: 10003880 ns | Xbuf:
4309276501100 ns | Delta xbuf: 10003990 ns
Current: 4309286547600 ns | Delta current: 9985450 ns | Xbuf:
4309286494650 ns | Delta xbuf: 9993550 ns
Current: 4309296544020 ns | Delta current: 9996420 ns | Xbuf:
4309296492350 ns | Delta xbuf: 9997700 ns
Current: 4309306546550 ns | Delta current: 10002530 ns | Xbuf:
4309306494190 ns | Delta xbuf: 10001840 ns
Current: 4309316545080 ns | Delta current: 9998530 ns | Xbuf:
4309316494020 ns | Delta xbuf: 9999830 ns
Current: 4309326540320 ns | Delta current: 9995240 ns | Xbuf:
4309326490960 ns | Delta xbuf: 9996940 ns
Current: 4309336539100 ns | Delta current: 9998780 ns | Xbuf:
4309336491210 ns | Delta xbuf: 10000250 ns
Current: 4309346541130 ns | Delta current: 10002030 ns | Xbuf:
4309346491060 ns | Delta xbuf: 9999850 ns
Current: 4309356540620 ns | Delta current: 9999490 ns | Xbuf:
4309356491050 ns | Delta xbuf: 9999990 ns
Current: 4309366540870 ns | Delta current: 10000250 ns | Xbuf:
4309366491000 ns | Delta xbuf: 9999950 ns
Current: 4309376539050 ns | Delta current: 9998180 ns | Xbuf:
4309376490710 ns | Delta xbuf: 9999710 ns
Current: 4309386540010 ns | Delta current: 10000960 ns | Xbuf:
4309386491770 ns | Delta xbuf: 10001060 ns
Current: 4309396540120 ns | Delta current: 10000110 ns | Xbuf:
4309396489810 ns | Delta xbuf: 9998040 ns
Current: 4309406538270 ns | Delta current: 9998150 ns | Xbuf:
4309406490520 ns | Delta xbuf: 10000710 ns
Current: 4309416542440 ns | Delta current: 10004170 ns | Xbuf:
4309416492100 ns | Delta xbuf: 10001580 ns
Current: 4309426541380 ns | Delta current: 9998940 ns | Xbuf:
4309426491610 ns | Delta xbuf: 9999510 ns
Current: 4309436540760 ns | Delta current: 9999380 ns | Xbuf:
4309436491200 ns | Delta xbuf: 9999590 ns
Current: 4309446540220 ns | Delta current: 9999460 ns | Xbuf:
4309446490910 ns | Delta xbuf: 9999710 ns
Current: 4309456541300 ns | Delta current: 10001080 ns | Xbuf:
4309456489930 ns | Delta xbuf: 9999020 ns

```

2.2 Mode sous surcharge

```
evl-de1 ~ # taskset -pc 1 $$  
evl-de1 ~ # stress --cpu 8 --io 4 --vm 1 --timeout 10s
```

```
evl-de1 ~ # ./xbuf 20  
Current: 4446600575230 ns | Delta current: 0 ns | Xbuf: 4446600515030 ns |  
Delta xbuf: 0 ns  
Current: 4446610565230 ns | Delta current: 9990000 ns | Xbuf:  
4446610504090 ns | Delta xbuf: 9989060 ns  
Current: 4446630376770 ns | Delta current: 19811540 ns | Xbuf:  
4446630317800 ns | Delta xbuf: 19813710 ns  
Current: 4446630710000 ns | Delta current: 333230 ns | Xbuf: 4446630601390  
ns | Delta xbuf: 283590 ns  
Current: 4446640555350 ns | Delta current: 9845350 ns | Xbuf:  
4446640499150 ns | Delta xbuf: 9897760 ns  
Current: 4446650609410 ns | Delta current: 10054060 ns | Xbuf:  
4446650537040 ns | Delta xbuf: 10037890 ns  
Current: 4446660668650 ns | Delta current: 10059240 ns | Xbuf:  
4446660608970 ns | Delta xbuf: 10071930 ns  
Current: 4446671072090 ns | Delta current: 10403440 ns | Xbuf:  
4446671015870 ns | Delta xbuf: 10406900 ns  
Current: 4446680994680 ns | Delta current: 9922590 ns | Xbuf:  
4446680936640 ns | Delta xbuf: 9920770 ns  
Current: 4446690854520 ns | Delta current: 9859840 ns | Xbuf:  
4446690797580 ns | Delta xbuf: 9860940 ns  
Current: 4446700540580 ns | Delta current: 9686060 ns | Xbuf:  
4446700483750 ns | Delta xbuf: 9686170 ns  
Current: 4446710540390 ns | Delta current: 9999810 ns | Xbuf:  
4446710486360 ns | Delta xbuf: 10002610 ns  
Current: 4446720538570 ns | Delta current: 9998180 ns | Xbuf:  
4446720482880 ns | Delta xbuf: 9996520 ns  
Current: 4446730539850 ns | Delta current: 10001280 ns | Xbuf:  
4446730483670 ns | Delta xbuf: 10000790 ns  
Current: 4446740543560 ns | Delta current: 10003710 ns | Xbuf:  
4446740488360 ns | Delta xbuf: 10004690 ns  
Current: 4446750588130 ns | Delta current: 10044570 ns | Xbuf:  
4446750512290 ns | Delta xbuf: 10023930 ns  
Current: 4446760573140 ns | Delta current: 9985010 ns | Xbuf:  
4446760512420 ns | Delta xbuf: 10000130 ns  
Current: 4446770554760 ns | Delta current: 9981620 ns | Xbuf:  
4446770497050 ns | Delta xbuf: 9984630 ns  
Current: 4446780538960 ns | Delta current: 9984200 ns | Xbuf:  
4446780483180 ns | Delta xbuf: 9986130 ns  
Current: 4446790538910 ns | Delta current: 9999950 ns | Xbuf:  
4446790483510 ns | Delta xbuf: 10000330 ns
```

2.3 Comparaison

En mode normal, les mesures obtenues sont très régulières. Les différences de temps entre deux échantillons successifs restent proches de 10 ms, aussi bien du côté Linux (`Delta current`) que du côté EVL (`Delta xbuf`).

Les valeurs issues du cross-buffer sont légèrement en avance sur celles mesurées par Linux. Cela est normal, puisque les timestamps sont générés directement dans la tâche EVL avant d'être transmis à la tâche Linux via le cross-buffer.

Sous surcharge, le comportement reste globalement stable. Quelques variations plus importantes apparaissent néanmoins, notamment une période proche de 20 ms suivie d'une période très courte d'environ 0.3 ms. Cela indique qu'une activation a été retardée, puis immédiatement rattrapée lors de l'itération suivante.

Malgré cette surcharge importante du système, les mesures issues du cross-buffer restent très proches des 10 ms attendus. Cela montre que la tâche EVL exécutée sur le CPU isolé conserve un comportement fortement déterministe.

On peut ainsi constater que l'utilisation des cross-buffers permet d'échanger des données entre le domaine temps réel (out-of-band) et le domaine Linux classique (in-band) avec très peu d'impact sur les performances temps réel.

3 Proxy

Cette partie du laboratoire consiste à utiliser un *file proxy* afin d'enregistrer dans un fichier les valeurs produites par une tâche temps réel. La tâche EVL simule deux capteurs en générant deux signaux sinusoïdaux décalés dans le temps et d'amplitudes différentes. Les valeurs sont ensuite transmises au proxy, qui se charge de les écrire dans un fichier de journalisation.

L'intérêt du proxy est d'éviter qu'une tâche *out-of-band* effectue directement des appels système Linux classiques, comme `write()`, car ceux-ci pourraient introduire de la latence dans la partie temps réel.

Nous nous sommes demandés s'il fallait faire varier les courbes selon le temps et non selon un `t` variant avec la boucle. En nous référant par rapport à la donnée, les courbes semblent être trop parfaites pour dépendre du temps récolté via le timer.

3.1 Mode normal

En mode normal, le programme est exécuté sans charge particulière sur le système. Les valeurs des deux capteurs sont enregistrées dans le fichier `sensors.log`, puis affichées sous forme graphique avec le script fourni.

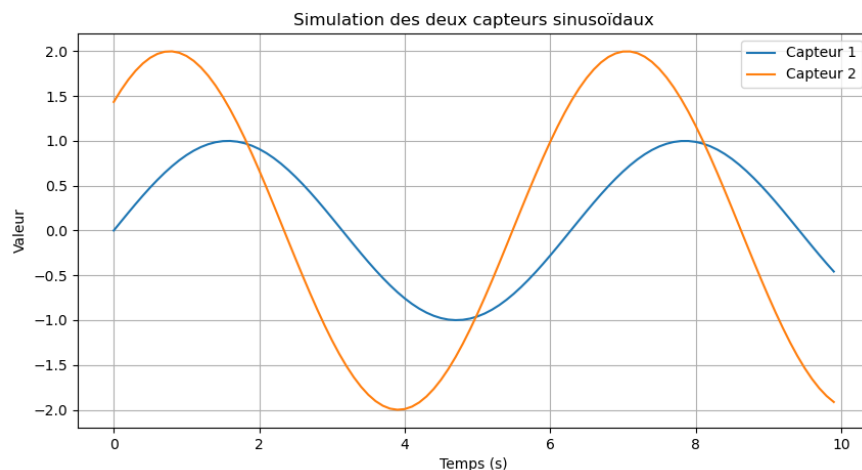


FIGURE 1 – Valeurs des capteurs en mode normal

On observe que les deux courbes correspondent bien à des signaux sinusoïdaux. Le second capteur possède une amplitude plus grande et un décalage temporel par rapport au premier capteur. Les données sont enregistrées correctement dans le fichier de log.

3.2 Mode sous charge

Le même programme est ensuite exécuté pendant que le système est chargé avec l'application **stress**. Le but est de vérifier si la charge du domaine Linux influence le comportement de la tâche temps réel ou l'enregistrement des données.

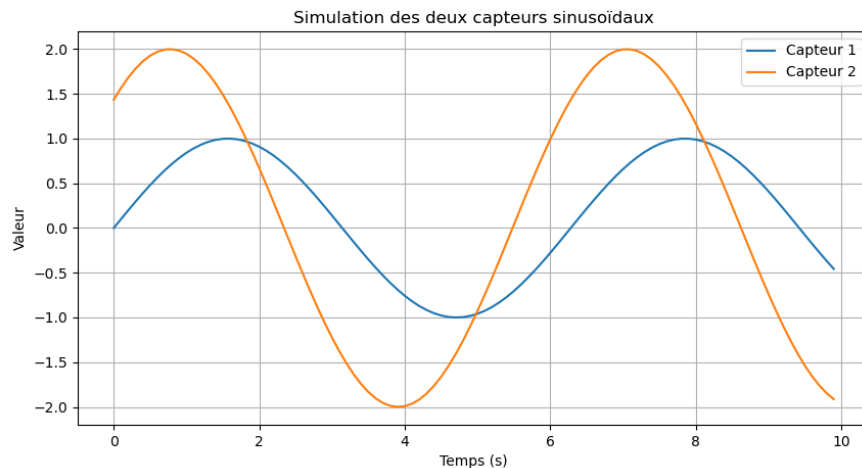


FIGURE 2 – Valeurs des capteurs en mode sous charge

On observe que les courbes obtenues sous charge sont très proches de celles obtenues en mode normal. Les deux signaux restent réguliers et aucune perturbation visible n'apparaît sur le graphe.

3.3 Comparaison

La comparaison entre les deux modes montre que l'utilisation de **stress** n'a pas d'effet visible sur les valeurs enregistrées. Cela indique que la tâche EVL continue à produire ses données périodiquement, même lorsque le domaine Linux est fortement sollicité.

Le proxy permet donc de séparer la partie temps réel, qui produit les valeurs des capteurs, de la partie Linux, qui s'occupe de l'écriture dans le fichier. Cette séparation évite de bloquer la tâche temps réel avec des appels système classiques et permet de conserver un comportement stable sous charge.

Dans cette expérience, les capteurs ont été simulés à partir d'une variable temporelle incrémentée à chaque période. Ce choix permet d'obtenir deux signaux réguliers, comme attendu par le script de visualisation fourni. Le test sous charge sert alors principalement à vérifier que la communication via proxy et l'écriture dans le fichier restent fonctionnelles même lorsque le système Linux est occupé.

4 Surveillance d'un système multi-tâches par Watchdog

```
evl-de1 ~ # ./flags
Worker 2 cycle 1, work time = 2503 ms
Worker 1 cycle 1, work time = 2000 ms
Worker 1 cycle 2, work time = 2007 ms
Worker 2 cycle 2, work time = 2510 ms
Watchdog cycle 1: OK
Worker 1 cycle 3, work time = 1993 ms
Worker 2 cycle 3, work time = 2496 ms
Watchdog cycle 2: OK
Worker 1 cycle 4, work time = 2000 ms
Worker 2 cycle 4, work time = 2503 ms
Watchdog cycle 3: OK
Worker 1 cycle 5, work time = 2007 ms
Worker 1 cycle 6, work time = 1993 ms
Worker 2 cycle 5, work time = 4000 ms
Watchdog cycle 4: OK
Worker 1 cycle 7, work time = 2000 ms
Watchdog cycle 5: TIMEOUT
-> Worker 2 missing
Worker 1 cycle 8, work time = 2007 ms
Worker 2 cycle 6, work time = 2496 ms
Watchdog cycle 6: OK
Worker 1 cycle 9, work time = 1993 ms
Worker 2 cycle 7, work time = 2503 ms
Watchdog cycle 7: OK
Worker 1 cycle 10, work time = 2000 ms
Worker 2 cycle 8, work time = 2510 ms
Watchdog cycle 8: OK
Worker 2 cycle 9, work time = 2496 ms
Watchdog cycle 9: OK
Worker 2 cycle 10, work time = 2503 ms
Watchdog cycle 10: TIMEOUT
-> Worker 1 missing
```


4.1 Analyse

Le programme utilise les *EVL flags* afin de synchroniser plusieurs tâches temps réel. Deux workers simulent des traitements périodiques avec une durée d'exécution différente et un léger jitter. À la fin de chaque cycle, chaque worker poste un flag afin d'indiquer au watchdog qu'il a terminé son traitement.

Le watchdog attend ensuite les flags des deux workers avec un timeout fixé à 3500 ms. Si les deux flags sont reçus avant l'échéance, le cycle est considéré comme valide et le watchdog affiche OK. Sinon, il déclenche une alerte et identifie le worker fautif grâce à l'état du groupe de flags.

Lors de l'exécution, le worker 2 simule volontairement une erreur au cycle 5 en augmentant son temps de traitement à 4000 ms. Cette durée dépasse le timeout du watchdog, qui détecte correctement l'erreur et indique :

```
Watchdog cycle 5: TIMEOUT  
-> Worker 2 missing
```

Après ce dépassement, les cycles suivants continuent normalement.

On remarque également un timeout au cycle 10 indiquant que le worker 1 est manquant. Ce comportement provient du fait que les workers ne terminent pas exactement au même instant et que la gestion des flags reste volontairement simple dans cette implémentation. Un flag retardé ou consommé trop tard peut ainsi provoquer un faux timeout sur le cycle suivant.

Ce laboratoire montre qu'un mécanisme de supervision basé sur les *EVL flags* permet de détecter efficacement les dépassements temporels dans un système temps réel multi-tâches.

5 Conclusion

Ce laboratoire nous a permis d'étudier plusieurs mécanismes temps réel sous Linux et Xenomai EVL. Nous avons pu comparer différentes approches pour la création de tâches périodiques et observer leur comportement dans des contextes normaux puis sous forte charge processeur.

L'utilisation d'EVL permet d'obtenir un comportement très déterministe grâce à l'exécution out-of-band et à l'utilisation d'un CPU isolé. Les mécanismes comme les cross-buffers, les proxies ou encore les flags offrent des outils efficaces pour développer des applications temps réel tout en conservant des échanges avec le monde Linux classique.

Enfin, le système de watchdog implémenté montre l'importance de la supervision dans les systèmes critiques, afin de détecter rapidement les dépassements de temps et les comportements anormaux des tâches.

6 Signatures

Yverdon-les-Bains le 19 mai 2026

Fabien Leger & Arnaut Leyre